A Hybrid Semantic Music Recommendation System Integrating Web Structure Mining and Social Sentiment Analysis

[Lê Văn Quân] **Course**:Web Mining - IT4868

Lecturer: Dương Quang Huy, PhD

Abstract

Traditional music recommendation system often rely heavily on collaborative filtering (requiring massive user logs) or low-level audio features,which fail to capture the semantic meaning and social perception of songs.This project proposes a Hybrid Semantic Recommendation System that overcomes these limitations by integrating Web Content Mining (Lyrics via LLM),Web Structure Mining (Song Graphs) and Social Sentiment Analysis (Youtube Comments).The system leverages Big Data technologies (Apache Spark, MongoDB) to process large-scale unstructured data and utilizes SBERT and RoBERTa models to extract deep semantic features. Experimental results will compare the proposed hybrid approach against traditional baseline models using metrics such as Precision@K, Diversity, and Silhouette Score.

1 Introduction

1.1 Motivation

The exponential growth of digital music platforms has made information overload a critical issue. While platforms like Spotify use advanced audio analysis, they often lack the "human touch"—the understanding of lyrical context and community sentiment. Furthermore, strict API limitations on user data access necessitate alternative approaches using Web Mining.

1.2 Problem Statement

Current systems face two main challenges:

- The Semantic Gap: Audio features (tempo, pitch) cannot fully represent complex human emotions (e.g., "nostalgia"or "heartbreak").
- The Cold-Start Problem: Difficulty in recommending new songs that lack user interaction history.

1.3 Objectives

- To develop a pipeline for mining unstructured data from the Web (Lyrics from Genius, Comments from YouTube).
- To apply Large Language Models (LLMs) for semantic embedding of lyrics.
- To construct a Song Graph and apply community detection algorithms to find latent music groups.
- To integrate Social Sentiment from YouTube comments to refine recommendations.

2 Proposed Methodology

The system architecture consists of four main modules:

2.1 Data Acquisition(Lakehouse Architecture)

We employ a multi-source data strategy:

- **Structured Data:** Metadata and Audio Features (Danceability, Energy, Valence) from the *Spotify Tracks Dataset* (Kaggle) to bypass API limitations.
- **Unstructured Text:** Lyrics scraped from *Genius/Musixmatch* using syncedlyrics.
- **Social Data:** User comments scraped from YouTube using youtube-comment-downloader.
- **Storage: MongoDB** is used to store unstructured documents (JSON), managed by **Apache Spark** for distributed processing.

2.2 Feature Engineering & NLP

We conduct a comparative analysis of text representation methods:

- **Baseline:** TF-IDF (Bag-of-Words).
- **Proposed:** Dense Vector Embedding using **SBERT (all-MiniLM-L6-v2)** to capture semantic similarity in lyrics.
- **Sentiment Analysis:** Utilizing pre-trained **RoBERTa** models to calculate the Social Valence Score from YouTube comments, capturing the "perceived mood"of the community.

2.3 Web Structure Mining

We construct a similarity graph $G = (V, E)$, where the nodes V represent songs and the edges E indicate semantic similarity greater than 0.8.

- **Algorithm: Louvain Community Detection** is applied to identify latent clusters (e.g., "Sad Indie", "Viral TikTok Hits").

- **Application**: Recommendations are boosted if candidate songs belong to the same community as the input song.

2.4 Hybrid Recommendation Engine

The final similarity score between Song A and Song B is computed as

$$\text{Score}(A,B) = w_1 \cdot \text{Sim}_{\text{Audio}}(A,B) + w_2 \cdot \text{Sim}_{\text{Semantic}}(A,B) + w_3 \cdot \text{Sim}_{\text{Social}}(A,B) + w_4 \cdot \delta(\text{Comm}_A, \text{Comm}_B). \quad (1)$$

where:

- $\text{Sim}_{\text{Audio}}$: Euclidean-distance-based similarity of audio features;
- $\text{Sim}_{\text{Semantic}}$: cosine similarity between SBERT embeddings;
- $\text{Sim}_{\text{Social}}$: similarity based on sentiment score alignment (positive/negative);
- $\delta(\text{Comm}_A, \text{Comm}_B)$: a Boolean function that equals 1 if Song A and Song B belong to the same community, and 0 otherwise.

3 Experimental Design

To evaluate the effectiveness of the proposed system, the following experiments will be conducted:

Experiment 1: Semantic Representation Efficacy

- **Objective:** Compare TF-IDF vs. SBERT in capturing lyrical meaning.
- **Metric:** Human Evaluation (Qualitative) on a subset of 50 songs.

Experiment 2: Impact of Social Sentiment

- **Objective:** Assess if adding YouTube comments improves recommendation relevance for "mood-based"queries.
- **Metric:** Sentiment Alignment Score (Correlation between Spotify Valence and YouTube Sentiment).

Experiment 3: System Performance (Big Data)

- **Objective:** Compare data processing speed between Pandas (Single-node) and PySpark (Distributed).
- **Metric:** Execution time (seconds) vs. Dataset size (10k, 50k, 100k rows).

4 Evaluation Metrics

Since real-time user logs are unavailable, intrinsic evaluation metrics will be used:

- **Precision@K**: Measures the proportion of relevant recommendations (same Genre/Artist) in the top K results.
- **Diversity Score:** Measures the dissimilarity between recommended items ($1 - \text{AvgSimilarity}$).
- **Silhouette Score:** Evaluates the quality of clusters formed by the Graph Mining module.

5 Project Timeline

- **Week 4:** Data Collection (Kaggle import, Web Scraping scripts for Lyrics/Comments).
- **Week 5:** NLP Implementation (SBERT Embedding, RoBERTa Sentiment Analysis).
- **Week 6:** Graph Mining (NetworkX graph construction, Louvain algorithm).
- **Week 7:** System Integration (Hybrid algorithm) & Big Data Optimization (Spark).
- **Week 8:** UI Development (Streamlit) & Final Report Writing.

6 Tools & Technologies

- **Languages:** Python 3.12.2
- **Big Data:** Apache Spark, MongoDB.
- **Libraries:** PySpark, Spotipy, Sentence-Transformers, NetworkX, Scikit-learn.
- **Web Mining:** Selenium, Youtube-comment-downloader, BeautifulSoup.
- **Interface:** Streamlit.